# nRQL Cheat Sheet

the RacerPro Query Language · verified on RacerPro 2.0 · github.com/lambdamikel/shacl-nrql-comparison

**Model:** a positive atom `(?x C)` binds `?x` only where the KB *entails* it (open world, DL reasoning); `neg` is *negation as failure* over *known* objects (closed world). **No unique-name assumption.**

## Server & protocol

```
LD_LIBRARY_PATH=.  ./RacerPro ⇒ TCP 8088.
Send "cmd\n"; read CR-terminated frame:
:answer <id> "res" "out" / :ok / :error
```

## KB setup

```
(full-reset)
(in-tbox NAME)              ; current TBox
(in-abox NAME [TBOX])       ; current ABox
(in-data-box NAME)          ; substrate
(in-mirror-data-box NAME)   ; auto-mirror
```

## TBox

```
(implies     Dog Animal)        ; Dog<=Animal
(equivalent  Mother
   (and Woman (some has-child Human)))
(disjoint    Man Woman)
(define-primitive-role has-child
   :inverse has-parent :transitive nil
   :domain Human :range Human)
(inverse     withdrawal deposit)
(implies-role deposit transaction)
(define-concrete-domain-attribute
   age :type integer)
```

## Concept expressions

```
(and ..) (or ..) (not C) (some R C) (all R C)
(at-least n R [C]) (at-most n R [C])
(exactly n R [C]) (one-of i ..)
(an attr)   ; "has some concrete value"
top  bottom
```

## ABox

```
(instance    alice Woman)
(related     alice junior has-child)
(constrained alice a-age age)
(constraints (equal a-age 50)) ; int->equal
(different-from k1 k2)         ; no UNA!
```

CD preds: int `(equal a v)`, real `(= a v)(> a v)`, string `(string= a s)`.

## Query forms

```
(retrieve  <head> <body>)
(retrieve1 <body> <head>)  ; for :reject
(tbox-retrieve <head> <body>)
(defquery NAME (vars) <body>)
```

## Variables

```
?x        ; ABox var, NON-injective
$?x       ; ABox var, INJECTIVE  (distinct!)
?*x  $?*x ; substrate node (non-inj / inj)
*ind      ; a named substrate individual
```

**NB:** reverse of the UG (RacerPro 2.0 was changed; `$?` is injective). Force distinctness explicitly — see Gotchas.

## Body atoms

```
(?x C)              ; concept (entailed)
(?x ?y R)           ; role
(?x ?y (inv R))     ; inverse role
(?x (has-known-successor R))
(TOP ?x) (BOTTOM ?x)
(same-as ?x ?y) | (= ?x ?y)
```

## Body constructors

```
(and B..) (union B..) (neg B) (inv B)
(project-to (?x..) B)  ; B may be a conjunction
(substitute (NAME args..))
;; "c's with no r-successor of type e"
;;  (complex projected body, negated):
(retrieve (?x) (and (?x c)
  (neg (project-to (?x)
         (and (?x ?y r) (?y e))))))
```

## Head projection

```
(?x)                       ; just the bindings
(?x (told-value (age ?x))) ; a told value
;; aggregation via head lambdas (MiniLisp)
```

## Killer idioms (live-verified)

```
;; sh:minCount 1  (= NOT EXISTS)
(retrieve (?x) (and (?x Person)
  (neg (project-to (?x) (?x ?y fn)))))
;; sh:class on a path  (no counterexample)
(retrieve (?x) (and (?x Parent)
  (?x ?y has-child) (neg (?y Person))))
;; sh:maxCount 1  (no UNA: over-reports)
(retrieve (?x) (and (?x Person)
  (?x $?a fn) (?x $?b fn)
  (neg (same-as $?a $?b))))
;; sh:disjoint  (shared value)
(retrieve (?x)
  (and (?x ?v likes) (?x ?v dislikes)))
```

## Data substrate / property graphs

```
(data-node acc1
  ((:amount 1000.0)(:currency $)) account)
(data-edge acc1 acc2
  ((:amount 9900.0)) deposit) ; edge props!
(node-label acc1) (edge-label acc1 acc2)
(retrieve (?*x)
  (?*x ((:amount (:predicate (> 1e4))))))
(retrieve (?*x ?*y)
  (?*x ?*y (:predicate (equalp deposit))))
(?*x (:predicate (search "Joh"))) ; substr
```

## Rules

```
(firerule
  (and (?x ?y has-child) (?y ?z has-child))
  ((related ?x ?z has-grandchild)))
; antecedent may use neg; consequent may
; (instance (new-ind tag ?x) C) -- create.
```

## Reasoning (what a validator can't)

```
(classify-tbox) (taxonomy)
(concept-subsumes? C D)  ; D<=C inferred
(concept-satisfiable? C)
(realize-abox)
(individual-instance? i C)
(individual-types i)
(concept-instances C)
(abox-consistent?)
```

## Query reasoning (the exotic part)

```
(prepare-abox-query (?x) (?x Mother)) ;:q-1
(query-consistent-p :query-1) ; ever nonempty?
(query-entails-p :query-1 :query-2) ; subsump
(query-equivalent-p :query-1 :query-2)
```

## OWL

```
(owl-read-file "/path/onto.owl")
(concept-instances #!:DogOwner) ; inferred
(individual-instance? #!:john #!:DogOwner)
```

## MiniLisp hatch (termination-safe!)

```
(evaluate (+ 1 2))    ; restricted evaluator
;; retrieve1 head with a
;;   (:lambda (x) .. :reject) = a filter
```

**Has** arith, `format`, `search`, `subseq`, `reduce`, `dotimes`, KB callbacks. **Lacks** recursion (aborted to ensure termination), `loop`, `require`, regex. Not Turing-complete — by design, for a decidable reasoner.

## Gotchas

- **No UNA**: `(at-most 1 R)` + 2 named fillers is *consistent* until `(different-from..)`.
- `(neg (?x ?y R))` ≠ `(neg (project-to (?x)(?x ?y R)))` — use the 2nd for "no R-successor".
- `neg` = failure over *known* objects, not classical negation, not "triple absent".
- `has-known-successor` sees *named* successors; an OWL existential's anonymous witness it won't.
- **Var inequality**: `(neg (same-as ?x ?y))` or `(neg (= ?x ?y))`. No `<>` for individuals; `different-from` is ABox-only. UNA toggling doesn't change query var-distinctness.